Exact Evaluation of Adaptive Prefix-Cache Continuations

Mohammad Ali Seyed Mahmoud
Georgia Institute of Technology
m.seyedmahmoud@gatech.edu

Abstract

In LLM serving with prefix caching, several request orders can incur the same immediate prefill work yet leave different cache states. We give an exact method to score a known, finite continuation from each state under least-recently-used eviction and entry-time charging of whole prefix paths. The continuation can select its next request using current hits. Equivalent starting states can share scores, and a sufficient persistence test can determine a score without replay. In one previously examined Mooncake tool/agent condition (128 blocks, decision output), the two shortcuts reduced full replays from 267,672 to 46,610. The complete-state and focused-state implementations used the same shortcuts and agreed on replay counts in each of eight conditions, as expected. The complete-state implementation used more mean native CPU in six of the eight. In the highlighted condition the focused-state implementation took 66.59 microseconds per decision versus 89.00 for the complete-state implementation. Local caller wall time fell 16.9%. The policy saved 0.204% of modeled prefill work at 128 blocks and 1.001% at 32 on Mooncake, and zero on two external episodes. We have not established an end-to-end benefit for an independent planner.

1 Introduction

Suppose two orders for the same ready requests incur the same prefill work and leave the same prefixes resident, but leave those prefixes in different recency orders. The next admission can evict a different prefix. A scheduler that simulates the work remaining from each state also has to account for how later requests are chosen. A rule that prioritizes cache hits can issue the same pending requests in a different order from each state. What computation can the two simulations safely share?

This methods and measurement report quantifies the cost of preserving every candidate score. The historical scheduling policy that motivates it saved little prefill work (Section 6.4). It makes three contributions.

- An exact cache-state equivalence and a sufficient persistence test for finite continuations whose next request can depend on observed hits.
- Complete-state and focused-state implementations checked against 90,954 reference scores, with the selected order and its full cache state verified on the archived workloads.
- A matched measurement of the replays eliminated by certification and score sharing, their overlap, and the CPU cost of propagating the state on the remaining replays.

2 Related Work

Mattson et al. used LRU stack positions to evaluate a scalar address sequence across capacities [1]. Touzeau et al. classified hits and misses along program control flow by tracking addresses younger

than a focal address [2]. In the terminology of Stock et al., a block is persistent when all its accesses in a scope incur at most one miss [3]. Our persistence test specializes this idea to a known starting state and an entire path charged at admission. Grouping states that preserve transitions and rewards is a general principle in state aggregation [4].

SGLang's RadixAttention reuses common prefixes and uses cache-aware request ordering [5]. Dexter et al. introduced k-LPM to trade prefix reuse against latency constraints [6]. PEEK uses shared prefixes in the waiting queue to guide both admission and eviction [7]. Liu et al. studied online replacement rules and offline references and released the FreeInference trace used in our ordering pilot [8]. Here the quantity evaluated exactly is the work of a specified continuation from a specified cache state.

3 System Model and Problem Formulation

A node identifies a **complete prefix**. Equal content blocks under different parents identify different nodes. A reachable state F is a prefix-closed list of at most C distinct nodes, in most-recent-first retention order. The cacheable part of a request is a shallow-to-deep path $P = (p_1, \dots, p_l)$ of distinct nodes, capped so $l \le C$. Hits are observed at entry. The path is then admitted under a single service stamp and protected during that admission. Its nodes are placed shallowest first, old copies are removed, and the remainder retains its order. Thus

$$c_P(F) = l - |P \cap F|, \qquad T_P(F) = \mathrm{take}_C(P \parallel (F \setminus P)) \,. \tag{1}$$

Set deletion in (1) preserves list order. A resident ancestor has been touched at least as recently as its descendants. The depth rule places ancestors ahead of descendants with the same stamp. Truncation from the end therefore removes eligible leaves, and the cap on l protects the entire admitted path. For an input of L positions and block width b, the actual prefill charge is $(L - bl) + b\, c_P(F)$. The first term covers all positions outside the reusable path, including the compulsory final token. Its sum is constant across orders serving the same requests.

Figure 1(a) illustrates why the evaluator must preserve both the entry-time charge and the cache transition.

A *cohort* is the group of ready requests considered in a single decision (up to four here). For a legal order a, let w_a be its immediate missing-block count and let its *exit*, F_a, be the complete cache state after serving it. The policy considers only $\mathcal{A}_0 = \arg\min_a w_a$. The *incumbent* is the order preferred by the immediate scheduler's deterministic tie rule.

A *continuation* consists of a finite pending workload and the rule that chooses its service order. Each *historical pair* comprises two consecutive completed cohorts. The last sixteen completed cohorts supply up to eight disjoint pairs $\mathcal{H}_j$, grouped from the most recent end, with the older cohort served first within each pair. In each cohort our adaptation of k-LPM alternates between the oldest pending request and the request with the longest currently cached prefix, resolves ties by arrival order and restarts with the oldest at the next cohort. Each pair starts afresh from F_a. With at least one pair the policy selects an order minimizing

$$\sum_{j=1}^{m} J(F_a, \mathcal{H}_j), \qquad a \in \mathcal{A}_0,$$

where J counts missing blocks. Ties retain the incumbent when it is a minimizer, then favor arrival-lexicographic order. Empty history returns the incumbent. A *witness* is the complete representative exit from which the evaluator can replay one class of equivalent candidates.

4 Exact Continuation Evaluation

4.1 Cache-State Equivalence

Let U cover every node that **any permitted branch** of a specified finite continuation can access. For each resident $x \in U$, $\Pi_U(F)$ records its identity and absolute, one-based rank $r_F(x)$. Unlisted labels in U are absent. The gaps in these ranks retain the number of intervening outsiders even when their identities are discarded.

Proposition 1. Fix the capacity, namespace, pending workload and metadata. Suppose the selector, its internal updates, and its stopping rule depend on cache state only through observations recoverable from Π_U, including the charge for a chosen request. For equally initialized controllers, equality of the initial keys implies identical actions, observed charges and total continuation work.

Proof. For any available path $P \subseteq U$, the key identifies its resident nodes. Prefix closure makes them an initial segment, giving the hit length and c_P. After admission, p_i has rank i. Write $r_F(p) = \infty$ for an absent node. The prospective new rank of an initially resident $x \in U \setminus P$ of rank r is

$$\tilde{r} = l + r - |\{p \in P : r_F(p) \le r\}|, \tag{2}$$

where the whole path moves ahead of x, and old path entries already ahead of x are removed once. The node survives exactly when $\tilde{r} \le C$. An absent, untouched tracked node stays absent. Thus each permitted action determines the same charge and successor key from either initial state. Equal observations give the same controller decision and internal update. Induction over the finite continuation proves the result. The selector in Section 3 satisfies the observation restriction.

Figure 1(b) shows that membership alone can lose information and that unequal ranks can nevertheless yield an equal score. The historical application knows U from completed requests. Using only the labels seen along one realized branch of a more general adaptive procedure could omit a possible access.

Relation to focused LRU. For each resident node x, let Y_x denote the set of nodes ahead of it in LRU order. Absence has its own marker. At admission, the implementation charges all path misses before updating recency. Each p_i acquires the preceding $i-1$ path labels as its younger set. An untouched resident x has prospective younger set $Y_x \cup P$ and survives precisely when its size is less than C. All these sets refer to the same candidate execution. Write $A_x = Y_x \cap U$ and $b_x = |Y_x \setminus U|$. On reachable states, $r_F(x) = 1 + |A_x| + b_x$. Conversely, the resident labeled ranks identify A_x as the tracked residents ahead of x and give $b_x = r_F(x) - 1 - |A_x|$. Absence markers agree. Since no outsider is touched, the atomic update for an untouched resident becomes $(A_x \cup P, b_x)$ before truncation, exactly the update in (2). This establishes mutual reconstruction for the **correlated** outsider-anonymized observations from one cache. Independently combining possibilities for different focal labels could produce an impossible joint order.

4.2 Cache Persistence

Proposition 2. Now require U to be the exact union of labels that will **all be touched** in the continuation. Define $u = |U|$, $h = |F \cap U|$, and

$$z = \begin{cases} \max_{x \in F \cap U} r_F(x) - h, & h > 0, \\ 0, & h = 0. \end{cases}$$

If $u + z \le C$, the exact missing-block count is $u - h$ for every permitted service order.

Proof. The deepest initially resident tracked label has $h-1$ tracked labels ahead of it. Its number of younger outsiders is therefore z, an upper bound for every initially resident tracked label. Until

such a label is first touched, admission moves only members of U forward. Before each truncation there can be at most z outsiders and $u-1$ other tracked labels ahead of it. It survives, including at equality with capacity. Once touched, a tracked label can have only other tracked labels ahead of it and likewise survives. Every initially absent label incurs one miss on first use, and all repeated uses hit. This reasoning includes partially filled caches, $h = 0$ and empty paths. For an empty continuation the charge is zero. Apply the formula to capped paths and add compulsory positions separately via (1).

This is a sufficient persistence certificate. Stock, Hahn and Reineke call a block persistent over a scope when its accesses together incur at most one cache miss. The proof here specializes the standard LRU stack-distance argument to a known starting cache and entry-charged, whole-path admissions. It certifies zero misses for the initially resident labels and exactly one for each initially absent label, yielding the total. It does not decide persistence whenever the bound fails [3, 1].

The coverage distinction is essential. A superset of possible accesses suffices for Proposition 1. An unvisited, initially absent label in that superset would be counted spuriously by Proposition 2. The frozen historical pairs serve every request. When the sufficient test fails, the evaluator can still recover the cost by exact replay or by reusing an equal projected score.

4.3 Score Reuse and Order Selection

First group immediately optimal orders that reach the same **complete** exit, keeping the representative preferred by the dispatch tie rule. For each historical pair, the key in Proposition 1 induces a further partition of those exits. Evaluate certified costs directly and reuse each unresolved class's simulated cost across its members. Add the score to each candidate's own sum. Partitions are pair-specific: two candidates can share a term for one pair and have different terms for another. If every candidate has the same term, decision-only evaluation can skip it. Absolute scoring computes that term once and restores it for every candidate.

These operations preserve the ordering of objective values, including ties. The evaluator commits the winning order's own complete exit. Figure 1(c) shows why sharing a scalar historical term does not authorize substituting a different complete state.

(a) Entry-time charging. Start $[b, a]$, where b is another root and x is a child of a. Admitting $[a, x]$ incurs one miss and leaves $[a, x]$. Scalar touches x then a reach the same exit in two misses because x first evicts a.

(b) Absolute ranks. Serve singleton roots c then a. From $[a, b]$ the cost is one miss. From $[b, a]$ it is two. Both starts contain only a from this continuation's support. If the sole request is a, both starts cost zero despite their different ranks.

(c) Committing the exit. Serve only a from $[x]$ or $[y]$. Both cost one miss and yield the same key on $\{a\}$. The complete exits $[a, x]$ and $[a, y]$ differ on the next real request for x.

Figure 1: Three capacity-two examples. Lists show the most recent prefix first.

5 Implementations

The complete-state implementation with both shortcuts is RANK-11. The focused-state implementation is FOCUSED. The two digits in RANK indicate whether persistence certification and score sharing, respectively, are enabled. Both implementations use the same candidate generator, certificate, score memo and tie rules.

Evaluator	Survival test	Score sharing	State on residual replay
FULL = RANK-00	Off	Off	Complete candidate exit
RANK-10	On	Off	Complete candidate exit
RANK-01	Off	On	Complete representative exit
RANK-11	On	On	Complete representative exit
FOCUSED	On	On	Tracked identities and their absolute ranks

FOCUSED adapts the focused-LRU construction of Touzeau et al. [2]. When replay is needed, it scans the witness to initialize the reduced state and advances tracked ranks after each admission. RANK-11 replays the same continuation on the complete witness. FULL and RANK-00 share one code path. The comparison times these two implementations by the same author.

Computational complexity. For a historical pair with E distinct eligible exits, full evaluation can require E replays. With both shortcuts, it needs one replay per distinct uncertified class whose cost is required. Candidate generation is common work. The implementation scans up to EC slots to identify varying ranks, charges certificate and key construction, and stores class scores in a pair-local linear memo. If L exits require key lookup and they form D unresolved classes, key comparisons alone can cost $O(LDC)$. When every exit is distinguishable and uncertified, classification adds work without saving replays. The focused kernel trades a scan to convert each residual witness for updates over tracked residents. The history index also records identities encountered over the episode, so a sixteen-cohort history does not bound its storage.

6 Experimental Evaluation

6.1 Experimental Setup

The matched experiment reused 56 Mooncake bins [9] after their scheduling outcomes were known: 28 per traffic type, with 1,500 conversation cohorts and 2,745 tool/agent cohorts per capacity. All methods received the same arrivals, bin resets, warm-up, 512-token blocks and current-order search. Each traffic, capacity, output and implementation combination had three fresh-process repetitions on one Apple M2. The repeats measure timing variation on identical inputs; they supply no independent workloads, held-out timing validation or cross-machine estimate. Absolute output includes the integer historical sum and preferred order for every distinct eligible full exit. Decision output contains the chosen order and complete exit.

6.2 Correctness

Synthetic verification covered 3,287 decision inputs, 9,304 literal focused updates and 4,000 checked native continuation trajectories. For two integration bins at both capacities, all-candidate scoring was independently checked against the stamped-leaf oracle. Across integration and measured bins, 8,766 cohort–capacity cases supplied 90,954 reference candidate scores. Every absolute timing cell reproduced the expected score vectors, and every cell reproduced the selected orders, committed complete exits and episode prefill. These finite checks test the implementation of the propositions.

6.3 Runtime and Replay Counts

Replay counts. Table 2 reconciles every pair–exit evaluation in the 128-block tool/agent decision condition. Skipping 2,648 common pair terms before scoring avoided 26,391 evaluations when weighted by the number of eligible exits. Adding certification to sharing eliminated 71,483 replays, and adding sharing to certification eliminated 68,095. Adding the two standalone savings would double-count

Table 1: Mean complete native CPU time (μs per cohort) over three repetitions. Timing includes parsing, candidate construction, historical evaluation and state maintenance. Lower is better. Bold marks the lowest mean per row. Ratio = FOCUSED/RANK-11.

Output	Traffic	Blocks	FULL	RANK-10	RANK-01	RANK-11	FOCUSED	Ratio
Decision	Conversation	128	269.94	172.05	**29.85**	29.94	30.02	1.003
Decision	Conversation	32	57.00	55.70	14.05	**13.77**	14.28	1.037
Decision	Tool/agent	128	281.63	146.02	156.37	89.00	**66.59**	0.748
Decision	Tool/agent	32	54.79	53.29	40.30	39.72	**33.69**	0.848
Absolute	Conversation	128	273.19	174.75	51.04	42.50	**37.65**	0.886
Absolute	Conversation	32	57.60	57.32	20.57	20.48	**18.54**	0.905
Absolute	Tool/agent	128	279.21	145.53	157.24	90.07	**67.78**	0.753
Absolute	Tool/agent	32	56.32	54.77	42.58	42.27	**35.89**	0.849

In repetition order the FOCUSED/RANK-11 ratios in the 128-block tool/agent decision condition were 0.758, 0.725 and 0.762.

81,484 replays. The two implementations with both shortcuts performed 46,610 replays in this condition, 82.6% fewer than FULL. Their replay counts agreed in each of the eight conditions, as expected from their shared logic. Complete-state propagation used more mean native CPU in six of the eight (Table 1).

Table 2: Historical pair–exit evaluations in the highlighted condition. Each row accounts for 267,672 evaluations as the sum of common terms skipped, direct scores, memo hits and residual replays. Checks are shown separately.

Method	Common skipped	Checks	Direct	Memo hits	Residual replays
FULL	0	0	0	0	267,672
Certification	0	267,672	152,967	0	114,705
Sharing	26,391	0	0	123,188	118,093
Both (each kernel)	26,391	241,281	139,871	54,800	46,610

$267{,}672 = 26{,}391 + 241{,}281$ $\quad$ $241{,}281 = 139{,}871 + 54{,}800 + 46{,}610.$

CPU and wall time. In that same cell, FULL to FOCUSED reduced mean complete native CPU by 76.4% (281.63 to 66.59 μs). At 32 blocks the corresponding tool/agent reductions were 38.5% for decisions and 36.3% for absolute scores. Comparing the two strong implementations at 128 blocks, the historical phase, including checks and sharing, took 38.42 μs for FOCUSED and 60.10 μs for RANK-11. Native work outside that phase accounted for 28.17 and 28.90 μs respectively. Complete native CPU fell 25.2%, while caller wall, including IPC and response decoding, fell 16.9% (133.35 to 110.80 μs). These clocks do not measure serving latency.

Effect of workload. For conversation decisions at 32 blocks, adding certification to sharing saved zero replays (151 either way). At 128 it reduced 223 to 156, with no mean CPU gain: RANK-01 took 29.85 μs and RANK-11 took 29.94 μs. RANK-11 and FOCUSED both took about 30 μs there and about 14 μs at 32 blocks. For absolute scores on conversation at 128 blocks, the common terms still have to be valued: adding certification to sharing reduced replays from 12,182 to 6,975 and complete native CPU from 51.04 to 42.50 μs (RANK-01 to RANK-11).

Earlier benchmark, different comparator, pooled traffic. The preceding experiment pooled the 4,245 measured Mooncake cohorts across both traffic types at 128 blocks. Mean native evaluator CPU fell from 283.03 μs with full-state replay without the combined shortcuts to 68.72 μs with the optimized

tie-only evaluator, a 4.12× evaluator speedup. The local caller ran 2.93× faster. Table 1 reports the later matched timing by workload.

6.4 Scheduling Results

Table 3 distinguishes exact evaluation of a historical objective from the scheduling value of that objective. On Mooncake the tie-breaker saved 478 blocks at capacity 128 and 2,459 at capacity 32 over the same 15,432 requests. All 128-block savings occurred on tool/agent traffic. On Qwen chat, 4,643 of 5,200 primary cohorts offered multiple immediately optimal exits. History always retained the incumbent, although two of three frozen random tie controls achieved lower episode work. On Qwen API, each selected request had its own chat root, precluding cross-request hits under the frozen lineage rule.

Table 3: Scheduling evidence across four studies. Capacity is measured in 512-token blocks unless otherwise stated. The Mooncake and Qwen percentages compare modeled prefill positions within each episode or pooled sample.

Study and trace	Capacity	Comparison	Outcome
Queue-aware eviction, Mooncake	128	Immediate search versus incumbent, root optimality bound	2,452/2,453 cohorts certified
LRU tie-breaker, Mooncake	128 / 32	History versus exact immediate ordering	0.204% / 1.001% saved
LRU tie-breaker, Qwen chat / API [10]	128	History versus exact immediate ordering, 20,000 requests each	0% / 0% saved
Ordering pilot, FreeInference [8]	4 pools, 28.3–282.7 GiB	Best tested LRU order versus FCFS, 3,000 requests	At most 1.323% saved, 0/36 reached 3%

The held-out FreeInference pilot allowed each request to start prefill up to 500 ms after its FCFS start. All nine configurations that qualified when the offline reference could also change eviction used the smallest pool (402 blocks, 28.3 GiB), the only tested pool that fits alongside the modeled BF16 weights on an 80 GB H100. In the 60%-utilization, 16-request queue condition, the reference order saved 0.591 percentage points beyond the best online order with LRU. Holding that order fixed and changing eviction added 7.174 points, for 7.765 points total with FCFS as the denominator. The additional saving is attributable to future-informed eviction. This feasible reference establishes an attained saving, with no proof of optimality or guarantee for an online policy. The full ordering pilot on GitHub gives the frozen protocol and per-configuration audit.

7 Discussion and Limitations

Both timing implementations were written by the same author and share code. Consequently the comparison measures the cost of two state representations within one implementation family. The timing bins were reused after their scheduling outcomes were known. Three fresh-process repetitions on one Apple M2 measure machine variation on identical inputs and provide no cross-machine estimate. Neither the evaluator benchmark nor the ordering pilot measured end-to-end serving latency. Mooncake reset each bin and warmed over its first quarter, while Qwen supplied continuous episodes with reconstructed chat lineages. Trace source, namespace, warm-up and episode boundaries therefore vary together.

The historical scheduler consumes only the winning action and its full exit. Bounds and early termination that return the same action and preserve its tie rule are eligible decision-only baselines.

The benchmark did not time such alternatives. A targeted search of published cache planners and LLM query optimizers looked for a caller requiring exact continuation costs from alternative cache states at one fixed capacity. Three near matches illustrate the distinction. WAKMan replays recent traffic to choose an eviction policy [11]. Optimal Prefix Hit Recursion scores shared leading fields in a proposed row layout [12]. KVSET derives hit rates across capacities from a single LRU history [13]. The screen found no qualifying independent caller to profile.

8 Conclusion

Under entry-charged whole-path LRU, the two propositions give an exact evaluator for known finite continuations whose selector observes recoverable cache hits. The implementations reproduced 90,954 reference candidate scores. In the 128-block Mooncake tool/agent decision condition, certification and score sharing required 82.6% fewer replays than FULL, and focused-state propagation used 25.2% less complete native CPU than complete-state propagation. This specialization offers limited incremental novelty over focused LRU. No deployment benefit has been demonstrated.

Artifact Availability

The eight timing cells, evaluator implementation, verification code, trace sources and build instructions are available in the public GitHub artifact (revision `7a2f9a970a`) at https://github.com/smahmoud66/prefix-cache-studies.

The manuscript is distributed under Creative Commons Attribution 4.0 International.

Tool use. OpenAI Codex assisted with preparing the manuscript and computational artifact and with running the release checks. The reported measurements come from the archived experiments.

References

[1] R. L. Mattson, J. Gecsei, D. R. Slutz, and I. L. Traiger. Evaluation techniques for storage hierarchies. *IBM Systems Journal*, 9(2), 78–117, 1970. doi:10.1147/sj.92.0078.

[2] V. Touzeau, C. Maïza, D. Monniaux, and J. Reineke. Fast and exact analysis for LRU caches. *Proceedings of the ACM on Programming Languages*, 3(POPL), 1–29, 2019. doi:10.1145/3290367.

[3] G. Stock, S. Hahn, and J. Reineke. Cache persistence analysis: Finally exact. *IEEE Real-Time Systems Symposium*, 2019. arXiv:1909.04374.

[4] R. Givan, T. Dean, and M. Greig. Equivalence notions and model minimization in Markov decision processes. *Artificial Intelligence*, 147, 163–223, 2003. doi:10.1016/S0004-3702(02)00376-4.

[5] L. Zheng et al. SGLang: Efficient execution of structured language model programs. *Advances in Neural Information Processing Systems*, 37, 2024. arXiv:2312.07104.

[6] G. Dexter, S. Tang, A. Fatahi Baarzi, Q. Song, T. Dharamsi, and A. Gupta. LLM query scheduling with prefix reuse and latency constraints. *Advances in Neural Information Processing Systems*, 38, 2025. arXiv:2502.04677.

[7] B. Xie, Z. Wang, M. Tanaka, and Z. Zheng. PEEK: Predictive queue-informed KV cache management for LLM serving. 2026. arXiv:2607.02525v2.

[8] Y. Liu, M. Yu, and J. Yang. When fancy eviction fails: Rethinking cache replacement for LLM prefix reuse. 2026. arXiv:2609.28870v3. FreeInference dataset, revision `ba89a343`.

[9] R. Qin, Z. Li, W. He, J. Cui, F. Ren, M. Zhang, Y. Wu, W. Zheng, and X. Xu. Mooncake: Trading more storage for less computation—a KVCache-centric architecture for serving LLM chatbot. *23rd USENIX Conference on File and Storage Technologies*, 155–170, 2025. Preprint: arXiv:2407.00079. Released traces, revision `3cca71d`.

[10] J. Wang, J. Han, X. Wei, S. Shen, D. Zhang, C. Fang, R. Chen, W. Yu, and H. Chen. KVCache cache in the wild: Characterizing and optimizing KVCache cache at a large cloud provider. *2025 USENIX Annual Technical Conference*, 465–482, 2025. arXiv:2506.02634. Qwen/Bailian traces, revision `5f7439c`.

[11] S. K. Morisetty, V. Lokegaonkar, S. M. Varghese, and W. Harris. WAKMan: Workload aware KVCache manager. 2026. arXiv:2610.36355v1.

[12] S. Liu et al. Optimizing LLM queries in relational data analytics workloads. *Proceedings of Machine Learning and Systems*, 7, 2025. Preprint: arXiv:2403.05821.

[13] L. Li, S. Wang, Z. Ruan, D. Li, and B. Gong. The KV cache working set: Online capacity planning for LLM inference systems. 2026. arXiv:2609.27746v1.